

HypatiaX Web Demo Setup Guide

Overview

This guide shows you how to use the **Web API** components to create a full-stack demo of HypatiaX.

File Structure

```
hypatiax/
├── demo/
│   ├── demo_web_api.py      # Flask REST API server
│   ├── engine.py           # Core processing engine
│   ├── ui.py               # Console UI components
│   ├── examples.py         # Example management
│   └── templates/
│       ├── demo.html       # Simple web interface
│       └── linkedin_visual_demo.html # Fancy demo
```

What Each File Does

demo_web_api.py - Flask API Server

Purpose: RESTful API backend that exposes HypatiaX functionality over HTTP

Key Features:

-  10 API endpoints (health, map, batch, examples, validate, etc.)
-  CORS enabled for frontend integration
-  Automatic model loading with fallback
-  Request statistics tracking
-  Error handling and logging
-  Serves HTML templates

API Endpoints:

Method	Endpoint	Description
GET	/	Serves demo.html
GET	/api/health	Health check + server status
POST	/api/map	Map description to formula
POST	/api/batch	Batch process multiple queries
GET	/api/examples	Get example queries
GET	/api/examples/categories	Get categories
POST	/api/validate	Validate generated formula

Method	Endpoint	Description
GET	/api/stats	Get server/engine statistics
GET	/api/methods	List available methods
GET	/api/test	Run test suite

demo.html - Simple Web Interface

Purpose: Clean, user-friendly web UI for testing HypatiaX

Features:

-  Real-time server status indicator
-  Quick example buttons
-  4 mapping method cards
-  Entity visualization with color coding
-  Formula display with metrics
-  Loading animations
-  Error handling

UI Components:

- Input field for natural language
 - Method selector (Vocab, Sentence, Regex, NER)
 - Example quick-load buttons
 - Results display with entities highlighted
 - Metrics dashboard (confidence, entities, time, method)
-



Quick Start (3 Steps)

Step 1: Install Dependencies

```
# Navigate to demo directory
cd hypatiax/demo

# Install Flask and dependencies
pip install flask flask-cors spacy pandas openpyxl

# Download spaCy model
python -m spacy download en_core_web_sm
```

Step 2: Start the Server

```
# Start with default settings
python demo_web_api.py

# Or with custom settings
python demo_web_api.py --host 0.0.0.0 --port 8000 --debug
```

Expected Output:

HypatiaX Web API Server	
URL:	http://127.0.0.1:5000
Status:	PRODUCTION / DEMO MODE
Models:	Loaded / Using rule-based

Available Endpoints:

GET /	- Demo web interface
GET /api/health	- Health check
POST /api/map	- Map description to formula
...	

Step 3: Open the Web Interface

Option 1: Server serves the page automatically
Open browser to: http://localhost:5000/

Option 2: Open demo.html directly
(Make sure server is running first)



Using the API

Example 1: Map a Description (cURL)

```
curl -X POST http://localhost:5000/api/map \
-H "Content-Type: application/json" \
-d '{
  "description": "sum of sales by region",
  "method": "vocab"
}'
```

Response:

```
{
  "success": true,
  "formula": "SUM([Sales])",
  "entities": [
    {
      "text": "sum",
      "label": "OPER",
      "start": 0,
      "end": 3,
      "confidence": 1.0
    },
    {
      "text": "sales",
      "label": "ARG",
      "start": 7,
      "end": 12,

```

```

        "confidence": 1.0
    },
    "confidence": 0.95,
    "processing_time": 12.5,
    "method": "vocab",
    "entity_count": 2
}

```

Example 2: Batch Processing (Python)

```

import requests

API_URL = "http://localhost:5000/api/batch"

data = {
    "descriptions": [
        "sum of sales",
        "average profit per product",
        "count total customers"
    ],
    "method": "vocab"
}

response = requests.post(API_URL, json=data)
result = response.json()

for r in result['results']:
    print(f"{r['description']} → {r['formula']}")

```

Example 3: Get Examples (JavaScript)

```

// Fetch all basic examples
fetch('http://localhost:5000/api/examples?category=basic')
    .then(response => response.json())
    .then(data => {
        console.log(`Found ${data.count} examples:`);
        data.examples.forEach(ex => {
            console.log(`  ${ex.description} → ${ex.expected_formula}`);
        });
    });

```

Example 4: Validate Formula

```

curl -X POST http://localhost:5000/api/validate \
  -H "Content-Type: application/json" \
  -d '{
    "description": "sum of sales",
    "expected_formula": "SUM([Sales])",
    "method": "vocab"
  }'

```

Response:

```

{
  "success": true,
  "match": true,

```

```
"generated_formula": "SUM([Sales])",  
"expected_formula": "SUM([Sales])",  
"confidence": 0.95  
}
```

Using the Web Interface

1. Enter Description

- Type your natural language query
- Or click one of the quick example buttons

2. Select Method

- **Vocab:** Fast, dictionary-based (recommended)
- **Sentence:** Pattern matching
- **Regex:** Rule-based extraction
- **NER:** ML-powered (requires trained models)

3. Generate Formula

- Click "🌟 Generate Formula"
- Watch the loading animation
- View results with color-coded entities

4. View Results

- **Entities:** Highlighted with labels (OPER, ARG, VERB, etc.)
 - **Formula:** Generated Tableau formula
 - **Metrics:** Confidence, entity count, processing time, method
-

Configuration Options

Server Configuration

```
# Custom host/port  
python demo_web_api.py --host 0.0.0.0 --port 8080
```

```
# Enable debug mode (detailed errors)  
python demo_web_api.py --debug
```

```
# All options  
python demo_web_api.py --host 0.0.0.0 --port 8080 --debug
```

Custom Model Paths

Edit `demo_web_api.py`:

```
engine = HypatiaXEngine(  
    desc_model_path='path/to/your/desc/model',  
    formula_model_path='path/to/your/formula/model'  
)
```

API Base URL in HTML

Edit `demo.html`:

```
// Change this line if server is on different port  
const API_BASE = 'http://localhost:5000/api';
```



Testing the API

Health Check

```
curl http://localhost:5000/api/health
```

Response:

```
{  
  "status": "online",  
  "version": "1.0.0",  
  "uptime_seconds": 125.4,  
  "models_loaded": true,  
  "mode": "production",  
  "stats": {  
    "total_requests": 15,  
    "successful": 14,  
    "failed": 1  
  }  
}
```

Run Test Suite

```
curl http://localhost:5000/api/test
```

Response:

```
{  
  "success": true,  
  "results": {  
    "vocab": {  
      "accuracy": 0.95,  
      "correct": 19,  
      "total": 20  
    },  
    "sentence": {  
      "accuracy": 0.88,  

```

```
        "correct": 17,  
        "total": 20  
    },  
    ...  
},  
"test_count": 20  
}
```

Deployment Options

Option 1: Local Development

```
# Start server  
python demo_web_api.py  
  
# Open browser  
http://localhost:5000/
```

Option 2: Network Access

```
# Bind to all interfaces  
python demo_web_api.py --host 0.0.0.0  
  
# Access from other devices  
http://YOUR_IP:5000/
```

Option 3: Production Deployment (Gunicorn)

```
# Install gunicorn  
pip install gunicorn  
  
# Run with 4 workers  
gunicorn -w 4 -b 0.0.0.0:5000 demo.demo_web_api:app
```

Option 4: Docker Deployment

Create Dockerfile:

```
FROM python:3.9-slim  
  
WORKDIR /app  
  
COPY requirements.txt .  
RUN pip install --no-cache-dir -r requirements.txt  
RUN python -m spacy download en_core_web_sm  
  
COPY . .  
  
EXPOSE 5000  
  
CMD ["python", "demo/demo_web_api.py", "--host", "0.0.0.0"]
```

Build and run:

```
docker build -t hypatiax-api .  
docker run -p 5000:5000 hypatiax-api
```



Troubleshooting

Server Won't Start

Issue: Address already in use

Solution:

```
# Find process using port 5000  
lsof -i :5000  
  
# Kill the process  
kill -9 <PID>  
  
# Or use different port  
python demo_web_api.py --port 8080
```

Can't Connect from Frontend

Issue: CORS errors in browser console

Solution:

- Ensure Flask-CORS is installed: `pip install flask-cors`
- Check server is running: `curl http://localhost:5000/api/health`
- Verify API_BASE URL in demo.html matches server address

Models Not Loading

Issue: "Using rule-based processing"

Solution:

```
# Check model paths exist  
ls ../data_spacy/queries/tableau/ner_tableau_desc  
  
# Update paths in demo_web_api.py if needed  
# Or continue with rule-based (works fine for demos)
```

Empty Results

Issue: No entities detected

Solution:

- Try different mapping methods

- Check example queries work first
 - Ensure description has recognizable terms (sum, average, etc.)
-

Integration Examples

Example 1: Python Script

```
import requests

class HypatiaXClient:
    def __init__(self, base_url='http://localhost:5000/api'):
        self.base_url = base_url

    def map_description(self, description, method='vocab'):
        response = requests.post(
            f'{self.base_url}/map',
            json={'description': description, 'method': method}
        )
        return response.json()

    def get_examples(self, category=None):
        params = {'category': category} if category else {}
        response = requests.get(
            f'{self.base_url}/examples',
            params=params
        )
        return response.json()

# Usage
client = HypatiaXClient()
result = client.map_description('sum of sales by region')
print(f"Formula: {result['formula']}")
```

Example 2: JavaScript/React

```
class HypatiaXAPI {
  constructor(baseURL = 'http://localhost:5000/api') {
    this.baseURL = baseURL;
  }

  async mapDescription(description, method = 'vocab') {
    const response = await fetch(`${this.baseURL}/map`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ description, method })
    });
    return await response.json();
  }

  async getExamples(category) {
    const url = category
      ? `${this.baseURL}/examples?category=${category}`
      : `${this.baseURL}/examples`;
    const response = await fetch(url);
  }
}
```

```
        return await response.json();
    }
}

// Usage
const api = new HypatiaXAPI();
const result = await api.mapDescription('average profit');
console.log(result.formula);
```



Performance Tips

1. Use Batch Processing

Instead of multiple single requests:

```
# Good: One batch request
result = requests.post('/api/batch', json={
    'descriptions': ['sum of sales', 'avg profit', 'count customers']
})

# Avoid: Multiple single requests
for desc in descriptions:
    requests.post('/api/map', json={'description': desc})
```

2. Cache Results

```
from functools import lru_cache

@lru_cache(maxsize=100)
def get_formula(description, method='vocab'):
    return requests.post('/api/map', json={
        'description': description,
        'method': method
    }).json()
```

3. Use Connection Pooling

```
from requests import Session

session = Session()
# Reuse connection for multiple requests
result = session.post('/api/map', json={...})
```



Next Steps

1. **Test Locally:** Start server and try the web interface
2. **Customize:** Edit demo.html to match your branding
3. **Integrate:** Use API in your application
4. **Deploy:** Put it on a server for team access

5. **Extend:** Add new endpoints for your specific needs

Quick Reference

Start Server

```
python demo_web_api.py
```

Access Web Interface

```
http://localhost:5000/
```

Health Check

```
curl http://localhost:5000/api/health
```

Map Description

```
curl -X POST http://localhost:5000/api/map \  
-H "Content-Type: application/json" \  
-d '{"description": "sum of sales", "method": "vocab"}'
```

View Stats

```
curl http://localhost:5000/api/stats
```

Summary

You now have a complete web-based demo system:

- ✓ **Backend:** Flask REST API with 10 endpoints
- ✓ **Frontend:** Clean HTML/JS interface
- ✓ **Integration:** Easy to connect from any language
- ✓ **Deployment:** Multiple options (local, network, Docker)
- ✓ **Testing:** Built-in test suite and validation

Ready to demonstrate HypatiaX to the world! 